DESIGN & DEVELOPMENT OF A REAL TIME FIRE DETECTION PROJECT

A

MINOR PROJECT REPORT

Submitted in partial fulfillment of the requirements

for the degree of

BACHELOR OF TECHNOLOGY

in

CSE-ARTIFICIAL INTELLIGENCE & DATA SCIENCE

By

GROUP NO. 06

Hemant Mohane	**0187AS231025**
Mohammed Zayed	**0187AS231033**
Rudraksh Sharma	**0187AS231049**
Abhishek Yadav	**0187AS231004**

Under the guidance of

Ms Madhuri Walia

Assi. Prof.

Department of CSE-Artificial Intelligence & Data Science

Sagar Institute of Science & Technology (SISTec) Bhopal (M.P.)

Approved by AICTE, New Delhi & Govt. of M.P.
Affiliated to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.)

Dec - 2025

Sagar Institute of Science & Technology (SISTec), Bhopal

Department of CSE-Artificial Intelligence & Data Science

CERTIFICATE

I hereby certify that the work which is being presented in the B.Tech. Minor Project Report entitled Design & Development of Real Time Fire Detector, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology, submitted to the Department of CSE with Artificial Intelligence & Data Science, Sagar Institute of Science & Technology (SISTec), Bhopal (M.P.) is an authentic record of my own work carried out during the period from July-2025 to Dec-2025 under the supervision of Ms. Madhuri Walia, Prof of CSE- AI&DS.

The content presented in this project has not been submitted by me for the award of any other degree elsewhere.

Hemant Mohane	***0187AS231025***
Mohammed Zayed	***0187AS231033***
Rudraksh Sharma	***0187AS231049***
Abhishek Yadav	***0187AS231004***

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Prof. Madhuri	**Dr. Vasima Khan**	**Dr. Manish Billore**
Project Guide	**HOD, CSE-AI&DS**	**Principal, SISTec**

ACKNOWLEGMENT

We would like to express our sincere thanks to **Dr. Manish Billore, Principal, SISTec** and **Dr. Swati Saxena, Vice Principal, SISTec** Gandhi Nagar, Bhopal for giving us an opportunity to undertake this project.

We also take this opportunity to express a deep sense of gratitude to **Dr. Vasima Khan, HOD**, **Department of CSE-Artificial Intelligence & Data Science** for her kindhearted support.

We extend our sincere and heartfelt thanks to our Guide, Ms. Madhuri Walia**,** for providing us with the right guidance and advice at the crucial junctures and for showing us the right way.

I am thankful to the **Project Coordinator, Prof. Ruchi Jain** who devoted her precious time in giving us the information about the various aspect and gave support and guidance at every point of time. I am thankful to their kind and supportive nature. His inspiring nature has always made my work easy.

I would like to thank all those people who helped me directly or indirectly to complete my project whenever I found myself in any issue.

TABLE OF CONTENTS

TITLE			PAGE NO.
Abstract			i
List of Abbreviation			ii
List of Figures			iii
List of Tables			vi
Chapter 1	Introduction		1
	1.1	About Project	1
	1.2	Project Objectives	2
Chapter 2	Software & Hardware Requirements		4
Chapter 3	Problem Description		7
Chapter 4	Literature Survey		12
Chapter 5	Software Requirements Specification		14
	5.1	Functional Requirements	14
	5.2	Non-Functional Requirements	15
Chapter 6	Software Design		17
	6.1	Use Case Diagram	17
	6.2	Project Flow Chart	19
Chapter 7	Machine Learning Module		23
	7.1	Data set Description	23
	7.2	Pre-processing Steps	24
	7.3	Data Visualization	25
	7.4	ML Model Description	26
	7.5	Result Analysis	26
Chapter 8	Front End Connectivity		28
Chapter 9	Coding		33
Chapter 10	Result and Output Screens		44
Chapter 11	Conclusion and Future Work		52
References			53
Appendix-1: Glossary of Terms			55
Project Summary			60-64

ABSTRACT

Early fire detection is crucial for preventing accidents, minimizing property losses, and ensuring safety in homes and public environments. Traditional fire detection systems rely on physical sensors such as smoke or heat detectors, which often respond late, offer limited coverage, and may generate false alarms. To overcome these limitations, this project presents a Real-Time Fire Detection System using computer vision and deep learning.

The proposed system utilizes the **YOLOv8** (You Only Look Once) object detection model, trained specifically to recognize flames in live video streams. The application captures frames from a webcam, processes them using **OpenCV,** and sends them to a Django backend, where the **YOLO model** performs fire prediction. The result—including whether fire is detected and the confidence percentage—is immediately returned to the frontend. A smooth, responsive user interface displays live video, while an audible alarm is triggered automatically when fire is detected.

To avoid false positives, the system incorporates a stability buffer, which evaluates multiple frames before confirming fire detection. This ensures accurate alerts even under fluctuating lighting conditions. The system is lightweight, real-time, and capable of running on standard hardware without requiring external sensors.

This project demonstrates how modern AI and computer vision can create an efficient, scalable, and reliable fire detection mechanism suitable for homes, offices, laboratories, and safety monitoring applications.

LIST OF ABBREVIATIONS

ACRONYM	FULL FORM
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheet
JS	JavaScript
UI/UX	User Interface/User Experience
API	Application Programming Interface
JWT	JSON Web Token
DL	Deep Learning

LIST OF FIGURES

FIG. NO.	TITLE	PAGE NO.
3.4.1	System Architecture of Fire Detection	09
6.1.1	Use Case Diagram	14
6.2.2	Data Flow Diagram (Level 0)	17
6.2.3	Data Flow Diagram (Level 1)	20
08.1	Home Page Interface	44
08.2	Fire Detection Result Display	45
10.4	Base64 Frame Conversion Process	46
10.5	YOLO Fire Detection Output	46

LIST OF TABLES

TABLE NO.	TITLE OF TABLE	PAGE NO.
2.1.1	Software Requirements (Python, Django, YOLO, OpenCV, MySQL)	4
2.2.1	Software Requirements (Python, Django, YOLO, OpenCV, MySQL)	5
5.1.1	Functional Requirements (Fire Detection, Real-Time Input, Alerts)	12
5.2.1	Non-Functional Requirements (Performance, Reliability, Accuracy)	13
7.1.1	Dataset Description (Fire vs. Non-Fire Images)	23
7.2.1	Pre-Processing Parameters (Resize, Normalization, Encoding)	24
7.4.1	YOLO Model Parameters (Confidence, IoU, Image Size)	26
7.5.1	Result Analysis Table (Precision, Recall, Accuracy)	26
8.4.1	Backend Connectivity (API Routes, POST Request Structure)	30
10.1.1	Output Screen Summary (Screenshots & Descriptions)	44

CHAPTER 1
INTRODUCTION

Fire accidents are one of the most common and dangerous hazards affecting homes, industries, forests, and public spaces. Traditional fire detection systems—such as smoke sensors and heat detectors—often respond late, rely on physical changes in the environment, and are prone to false alarms. With advancements in Artificial Intelligence (AI) and Computer Vision, early and accurate fire detection has become faster, more reliable, and more efficient.

This project focuses on developing an **AI-powered Real-Time Fire Detection System** using **YOLO (You Only Look Once)**, an advanced deep learning object detection model. The system processes live video frames captured from a camera, analyses them using a trained YOLO model, and detects fire with high accuracy. The backend is implemented using **Django**, and the frames are transmitted from the frontend to the server in base64 format. The server then performs inference using the ML model and returns the detection result instantly.

To enhance reliability, a **stability buffer mechanism** is used to prevent flickering results. Instead of reacting to a single frame, the system makes a stable decision based on the last several frames. This improves accuracy in challenging environments such as fluctuating light, camera noise, or small fire sparks.

The proposed solution offers a modern, intelligent approach to fire detection, capable of operating in real-time on low-cost hardware. It overcomes the limitations of traditional sensors and provides a strong foundation for safety systems in households, industries, parking areas, smart buildings, and surveillance networks.

Overall, this project demonstrates how AI, machine learning, and web technologies can work together to create a fast, efficient, and intelligent fire detection system capable of saving lives and property through early intervention.

1.1 ABOUT PROJECT

The Fire Detection System is an AI-driven real-time safety solution designed to identify fire at an early stage using deep learning and computer vision techniques. Traditional fire alarms rely on physical changes such as heat or smoke, which often leads to delayed response. To overcome this limitation, the proposed system uses a trained **YOLO object detection model** to analyse live camera frames and detect fire instantly.

The system uses a web interface where video frames are continuously captured and sent to the backend in base64 format. Django processes these frames, performs inference using the ML model, and returns the detection result within milliseconds. A stability buffer mechanism improves the accuracy by ensuring that decisions are made based on multiple consecutive frames.

The overall goal of the project is to reduce fire-related accidents by enabling fast, reliable, and automated fire detection.

1.2 PROJECT OBJECTIVE

- To design and implement a **real-time fire detection system** using deep learning and computer vision.
- To develop a backend using **Django** for processing camera frames and providing detection output.
- To train and deploy a **YOLO model** capable of identifying fire with high accuracy.
- To enable real-time communication between frontend and backend through base64-encoded frames.
- To reduce false positives by implementing a **stability buffer mechanism**.
- To provide a clean and interactive UI for displaying detection results and warnings.
- To create a scalable and practical detection system suitable for households, industries, and surveillance systems.

CHAPTER 2
SOFTWARE & HARDWARE REQUIREMENTS

2.1 SOFTWARE REQUIREMENTS

Table 2.1.1 Software Requirements

Category	Requirement	Purpose
Programming Languages	Python	Used for developing the YOLO fire detection model and backend logic in Django.
Frameworks & Libraries	JavaScript, HTML, CSS	Used for creating the real-time camera streaming frontend interface.
	Django	Backend web framework for handling POST requests, routing, and serving detection results.
	Django REST Framework	Handling API endpoints and structured communication between frontend and backend.
	YOLO (Ultralytics)	Deep learning framework for real-time fire detection.
	OpenCV	Capturing frames, preprocessing images, decoding base64 frames.
	NumPy	Array operations and efficient image processing.
	Base64	Encoding and decoding image frames sent from the browser.
Database	SQLite / MySQL	Storing logs, user details, alert history, and detection records (if required).
Browser Tools	Chrome / Edge	For accessing the streaming page and testing real-time dete
Development Tools	Visual Studio Code	Used for writing and managing Django, Python, and frontend code.
	Google Colab	Used for training and testing the YOLO model with GPU support.

		resources.
Hosting Platform	AWS / Heroku / PythonAnywhere	Hosting the Django server for online monitoring (optional).

2.2 HARDWARE REQUIREMENTS

Table 2.2.1 Hardware Requirements

Usage	**Requirement**	**Purpose**
Development Machine	Processor: Intel i5 / Ryzen 5 or higher	For smooth development, model training, and running Django server.
	RAM: 8 GB or higher	To load YOLO model efficiently and process real-time frames.
	GPU (Optional): NVIDIA GTX 1050 / RTX series	Speeds up model training and faster inference.
	Storage: 256 GB SSD	For storing dataset, model weights, logs, and project files.
Deployment (Server)	Processor: Intel Xeon or equivalent	Handles multiple requests and real-time frame processing.
	RAM: Minimum 4 GB	Suitable for lightweight YOLO-based inference.
	GPU (Optional)	Needed only for high-speed detection or multi-camera setups.
	Storage: 50–100 GB	Stores logs, model files, and detection history.
End User Device	Smartphone / Laptop / PC	Allows users to access real-time fire detection dashboard.
	Browser Support: Chrome / Firefox / Edge	Provides full functionality of the live streaming UI.

CHAPTER 3
PROBLEM DESCRIPTION

3.1 PROBLEM STATEMENT

Fire hazards pose a serious threat to human life, property, and the environment. Traditional fire detection systems such as smoke detectors, heat sensors, and alarm circuits rely on physical changes in the surroundings (temperature rise, smoke density), which often results in delayed detection. These systems also generate false alarms when triggered by dust, fumes, steam, or strong lights.

There is a need for a fast, accurate, and intelligent system that can detect fire **directly from visual input** rather than waiting for environmental changes.

Problem:
"How can we develop a real-time, camera-based fire detection system that uses deep learning to identify fire reliably, reduce false alarms, and provide immediate alerts?"

3.2 CHALLENGES IN THE CURRENT SYSTEM

- **Delayed Detection**
Traditional fire alarms activate only after significant smoke or heat is produced, reducing response time during emergencies.

- **High False Alarm Rate**
Smoke sensors can be triggered by dust, fog, steam, incense sticks, or cooking smoke, making them unreliable.

- **Limited Coverage Range**
Most smoke detectors cover only a small indoor area and cannot monitor large spaces like warehouses, parking lots, or open areas.

- **No Visual Confirmation**
Current systems cannot verify whether the detected smoke/heat actually indicates real fire.

- **No Remote Monitoring**
Basic systems do not provide remote monitoring or detection through mobile devices or web dashboards.

- **Lack of Intelligent Decision-Making**
Traditional systems do not use AI, meaning they cannot analyze patterns or distinguish between false and real fire events.

- **Inability to Handle Outdoor Environments**
Weather, lighting, and environmental noise cause major inaccuracies in conventional systems.

3.3 PROPOSED SOLUTION

To address these issues, this project proposes a **Real-Time Fire Detection System using YOLO and Django**, capable of identifying fire directly from live camera streams with high accuracy.

Key Components of the Proposed Solution

1. **AI-Based Fire Detection (YOLO Model)**
   A deep learning-based YOLO model is trained to identify fire from video frames in real time.
2. **Live Camera Streaming**
   Frontend captures frames and sends them to the backend in base64 format for continuous monitoring.
3. **Backend Inference (Django)**
   The backend receives frames, decodes them, runs YOLO inference, and sends detection results instantly.
4. **Stability Buffer Logic**
   To avoid frame-by-frame flickering, the system takes decisions based on several consecutive frames.
5. **Real-Time Alerts**
   Alerts are displayed on the UI when fire is detected.
6. **Scalable Deployment**
   The system can run on local devices, servers, or cloud environments for large-scale monitoring.

Benefits of the Proposed Solution

- Faster and earlier fire detection
- Reduced false alarms
- Real-time visual confirmation
- Works both indoors and outdoors
- Easily deployable with CCTV cameras
- Cost-effective and scalable

3.4 WORKFLOW OF AI BASED INTERVIEW SYSTEM

The complete workflow of the system follows these steps:

1. **Live Video Capture**
   The user's browser or CCTV camera stream sends continuous frames to the system.

2. **Frame Encoding**
   Each frame is converted to **base64 format** and sent via POST request to the Django backend.

3. **Backend Processing**
   Django receives the frame, decodes it into an image, and forwards it to the YOLO model for prediction.

4. **YOLO Prediction**
   The trained model analyses the frame and detects whether fire is present.

5. **Stability Buffer Application**
   Multiple recent detection results are stored to ensure a stable and reliable output.

6. **Response Back to UI**
   The backend returns JSON output: fire status (0/1) and confidence score.

7. **UI Alert Display**
   If fire is detected consistently, the system displays:
   "Fire Detected – Warning!"

8. **Optional: Notification System**
   Email/SMS/Alarm can be triggered for emergency response.

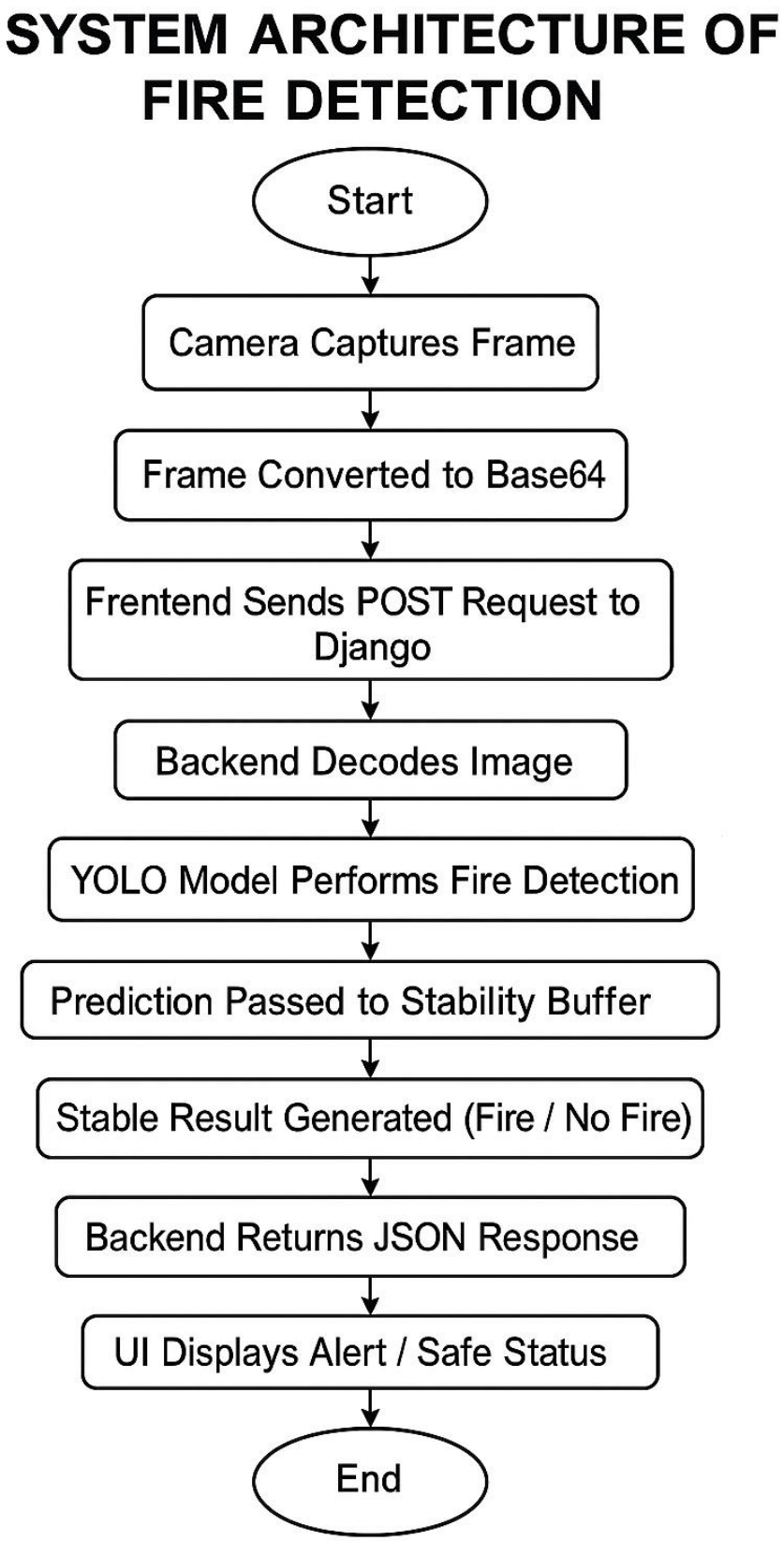


Figure 3.4.1 Work Flow Diagram

CHAPTER 4

LITERATURE SURVEY

This chapter reviews prior work and the current state of the art relevant to a **Real-Time Fire Detection System** using computer vision and deep learning (YOLO) integrated with a web backend (Django). The survey covers traditional sensor-based methods, classical computer-vision approaches, deep-learning solutions (with focus on YOLO-family models), evaluation practices, public datasets, and gaps that motivate this project.

4.1 Introduction

Fire detection has historically relied on physical sensors (smoke, heat, flame sensors). In recent years, camera-based detection using image processing and machine learning has gained traction because it can provide visual confirmation, cover wide areas, and enable remote monitoring. This literature review summarizes strengths and weaknesses of different approaches and positions YOLO + Django based real-time detection as the chosen approach for this project.

4.2 Traditional Sensor-Based Methods

Overview: Conventional systems primarily use smoke detectors (ionization or photoelectric), heat detectors, and flame sensors.

Advantages

- Mature, inexpensive, and easy to install.
- Low latency for localized detection (sensor detects local change quickly).

Limitations

- **Delayed detection** in many cases (need sufficient smoke/heat accumulation).
- **High false alarms** (cooking smoke, steam, dust, aerosols).
- **Limited coverage**: sensors cover localized areas only; many sensors needed for large areas.
- **No visual confirmation**: cannot verify cause or location without camera.

These limitations motivate visual detection methods that operate from cameras to provide earlier and more context-aware alerts.

4.3 Deep Learning Approaches

Deep learning models—particularly CNNs and single-shot detectors—have dramatically improved fire detection reliability.

Categories

- **Frame-level classification**: CNN classifies full image as fire/no-fire.
- **Region/proposal-based detection**: Faster R-CNN style approaches produce bounding boxes for flames.
- **Single-shot detectors**: YOLO family (YOLOv3/YOLOv5/YOLOv8), SSD — balance speed and accuracy and are suitable for real-time detection.

Advantages

- Learn discriminative features directly from data (color, texture, motion cues represented implicitly).
- Better generalization across varied scenes and lighting conditions.
- Provide bounding boxes for localization (useful for visual confirmation and downstream tasks).

Considerations

- Require labelled datasets (bounding boxes), more computation (GPU desirable), and careful training to avoid overfitting or class imbalance issues.

4.3 YOLO (You Only Look Once) for Fire Detection

Why YOLO?

- **Speed**: Real-time inference (single forward pass) — critical for live monitoring.
- **Accuracy**: Modern YOLO versions (v4/v5/v7/v8) achieve strong precision/recall for object detection tasks.
- **Simplicity**: Easy to deploy as a single model with bounding-box outputs.

Typical pipeline using YOLO

1. Collect annotated fire/non-fire images (with bounding boxes for flames).
2. Preprocess: resize, augment (flip, brightness, blur), balancing classes.
3. Train YOLO with appropriate loss, IoU, confidence thresholds.
4. Deploy model for inference; apply temporal stability/filtering to remove flicker.

Best practices in literature

- Use augmentations to improve robustness (lighting, scale, noise).
- Fine-tune pre-trained weights rather than training from scratch if dataset is small.
- Post-processing: apply temporal smoothing (e.g., buffer of recent detections) to reduce false positives due to transient visual artifacts.

CHAPTER 5
SOFTWARE REQUIREMENTS SPECIFICATION

1.1 FUNCTIONAL REQUIREMENTS

Functional requirements specify the key operations and behaviors the system must perform to achieve its goals.

Table 5.1.1 Functional Requirements

FR No.	Functional Requirement	Description
FR-1	Live Camera Capture	System must capture video frames from webcam/CCTV in real time.
FR-2	Convert Frame to Base64	Each frame must be converted to base64 format before sending.
FR-3	Send Frame to Backend	Frontend must send base64 frame through POST request (JSON).
FR-4	Decode Frame (Backend)	Django server must decode base64 image into OpenCV format.
FR-5	YOLO Fire Detection	Server must run YOLO model to detect Fire/No-Fire.
FR-6	Apply Stability Buffer	Backend must use last 10 frames to generate stable detection result.
FR-7	Return JSON Response	Server must respond with `{fire, conf, camera_id}` for each frame.
FR-8	Real-Time UI Display	Frontend must show Safe / Fire Detected alert.
FR-9	Log Detection Events	System must save detection logs (timestamp, fire, confidence).
FR-10	Admin Panel	Admin must configure model, thresholds, camera settings.
FR-11	Multi-Camera Support	System must handle multiple camera inputs (separate buffers).
FR-12	Error Handling	System must return proper error messages for invalid frames or requests.

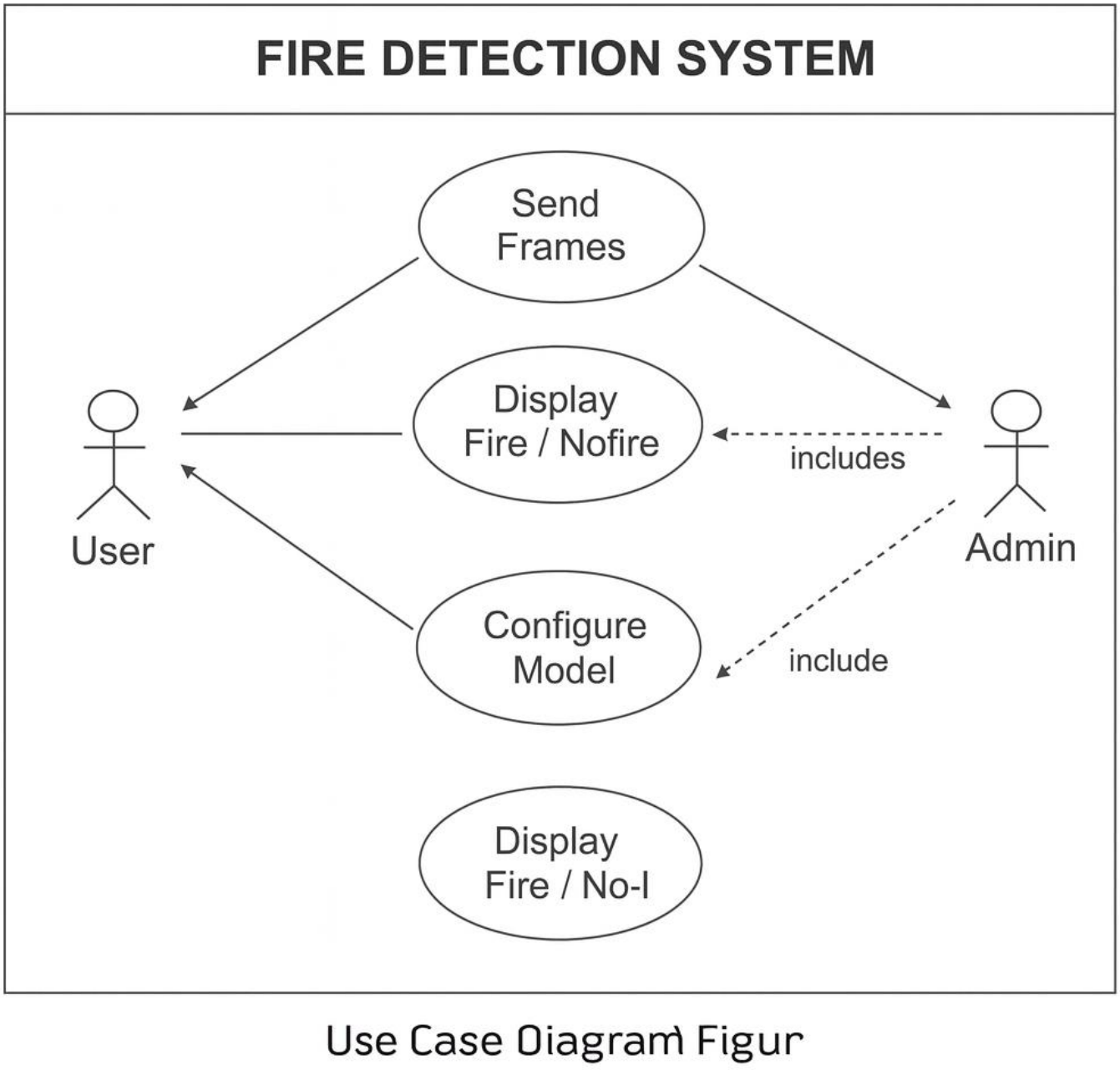
1.2 NON-FUNCTIONAL REQUIREMENTS

Table 5.1.1 Functional Requirements

NFR No.	Non-Functional Requirement	Description
NFR-1	Performance (Latency)	Detection pipeline must run within 300–500 ms per frame.
NFR-2	Accuracy	Model must achieve ≥ 85% precision and ≥ 80% recall.
NFR-3	Reliability	System must run continuously with ≥ 99% uptime.
NFR-4	Security	Admin pages must require login; use HTTPS in deployment.
NFR-5	Privacy	Camera frames must not be shared; store only when required.
NFR-6	Scalability	Server must support 2–5 camera streams depending on hardware.
NFR-7	Maintainability	Code must be modular, well-commented, and easy to update.
NFR-8	Portability	System must run on Windows/Linux and support cloud hosting.
NFR-9	Resource Limits	Backend must block frames > 5 MB (HTTP 413).
NFR-10	Usability	UI must show clear alert messages and status indicators.

CHAPTER 6
SOFTWARE DESIGN

6.1 USECASE DIAGRAM



Use Case Diagram Figur

Figure 6.1.1: Use Case Diagram

1. User

The *User* is the person operating the fire detection interface.
This actor performs the following actions:

- Starts the camera stream
- Sends frames to the backend
- Views the fire/no-fire results in real time
- Gets alerts if fire is detected

The User interacts only with the **frontend UI**.

2. Admin

The *Admin* is responsible for controlling the backend system settings.
The Admin mainly performs:

- Configuring model settings
- Updating model thresholds
- Monitoring the system
- Reviewing logs
- Managing camera IDs and detection parameters

The Admin has higher privileges than the User.

Use Cases (Oval Shapes)

1. Send Frames

This describes how the User's device:

- Captures a live camera frame
- Converts it to base64
- Sends it to the Django backend using a POST request

This is the starting point of the whole detection workflow.

2. Display Fire / No-Fire

This use case represents:

- How the system returns detection results
- How the UI displays the results
- How the alert is shown if fire is detected

This allows the User to visually confirm the presence or absence of fire.

3.Configure Model

This use case is **only for Admin**.

It includes:

- Adjusting detection thresholds
- Uploading new model weights
- Changing stability buffer size
- Managing camera settings
- Modifying processing FPS

This ensures the system can be tuned for accuracy and performance.

6.2 DATA FLOW

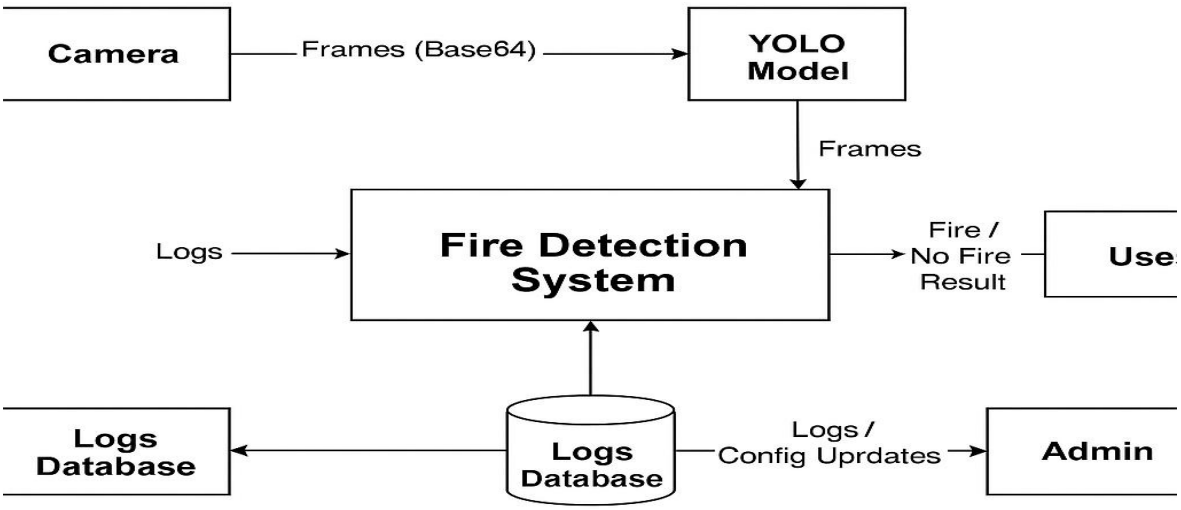


Figure 6.2.2: Data Flow Diagram level 0

1. Camera

The camera is an external entity that continuously captures live video frames.

Data Sent:

Frames (Base64 encoded)
The browser converts each frame into Base64 format before sending them to the YOLO model.

This ensures:

- Easy transfer over HTTP
- Compatibility with JSON-based API communication

2. YOLO Model

The YOLO (You Only Look Once) model receives Base64 frames from the camera and performs real-time fire detection.

YOLO Processes:

- Decodes images
- Runs the fire detection model
- Identifies “Fire” vs “No Fire”
- Generates bounding boxes and confidence values

3. Fire Detection System (Main Processing Block)

This is the core of the entire system.

Responsibilities:

- Receives fire detection results from YOLO
- Applies stability buffer (to avoid flicker or false positives)
- Generates the **final stable result**
- Sends result to the user
- Stores detection logs

4. User

The User is the person viewing the results from the frontend interface.

User Receives:

“Fire Detected” alert
“No Fire” (Safe) status
Confidence percentage

The user depends entirely on the processed result generated by the Fire Detection System.

5. Logs Database

This stores all detection-related information for future use.

Data Stored Includes:

- Timestamp of detection
- Camera ID
- Fire/No Fire result
- Confidence score
- Optional snapshot image

Purpose of Database:

- System monitoring
- Dashboard reporting

6. Admin

The admin monitors the system and updates configuration settings.

Admin Can:

- View detection logs
- Analyse fire detection accuracy
- Adjust thresholds (confidence, buffer size)
- Update YOLO model weights
- Manage camera configurations

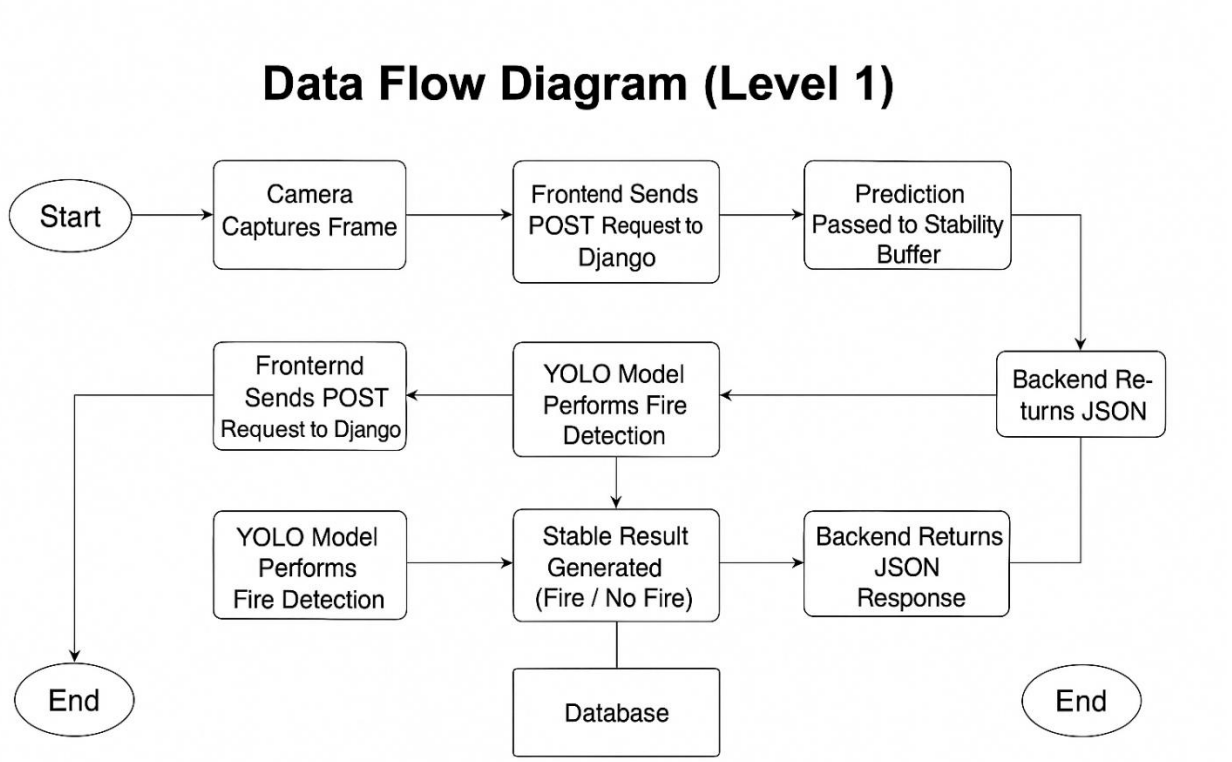


Figure 6.2.3: Data Flow Diagram level 0

1. Start

The process begins when the user activates the camera from the frontend.

---

2. Camera Captures Frame

The camera continuously captures video frames.
Each frame is then sent to the frontend for processing.

Data Flow:
Raw Frame (Image)

---

3. Frontend Sends POST Request to Django

The captured frame is:

- Converted into **Base64** string
- Embedded in a JSON object
- Sent to the Django backend using a POST API call

Data Flow:
`{frame: "base64string...", camera_id: "cam01" }`

---

4. YOLO Model Performs Fire Detection

Once the backend receives the frame:

- It decodes the Base64 image
- Sends the image to the YOLO model
- YOLO returns prediction results (Fire/No Fire + confidence)

Data Flow:
Detection Output (class + confidence)

---

5. Prediction Passed to Stability Buffer

To avoid flickering or false positives, the system uses a **stability buffer**:

- Maintains last 10 predictions
- Calculates stable output
- Only triggers "Fire Detected" if enough frames confirm fire

Purpose:
✔ Makes system stable and reliable
✔ Reduces false alarms from momentary noise

---

6. Stable Result Generated (Fire / No Fire)

The system now computes a final, stable decision:

- **Fire Detected**
- **No Fire**

This decision is what gets displayed to the user and stored in the logs.

7.Backend Returns JSON Response

After computing the stable result, the backend sends a JSON response back to the frontend containing:

```
{
  "fire": 0 or 1,
  "conf": 0-100,
  "camera_id": "cam01"
}
```

Data Flow:
Stable Detection Result

---

8. Frontend Receives JSON Response

The frontend UI receives the result and updates the display:

Shows “Fire Detected” alert
Shows Safe/No Fire status
Shows confidence percentage

---

9. Database Stores Result

Each detection event is saved to the database with:

- Timestamp
- Fire/No Fire
- Confidence
- Camera ID

Purpose:
Logs for admin
Performance analysis
Future improvements

---

10. End

The process repeats continuously for every new frame captured from the camera.

CHAPTER 7
CODING

9.1 STRUCTURE OF PROJECT

```
FIRE_DETECTOR
│
├── Fire_detector
│   ├── __pycache__
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
│
├── Home
│   ├── __pycache__
│   ├── migrations
│   │   ├── __pycache__
│   │   ├── 0001_initial.py
│   │   └── __init__.py
│   │
│   ├── templates
│   │   └── Home
│   │       └── home.html
│   │
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
│
├── ML_Model
│   └── fire_model.pt
│
├── static
│   ├── alarm.mp3
│   ├── fire_bg.jpg
│   ├── hemant.jpg
│   └── maka-bhosda-aag-meme-voicemod.mp3
│
├── db.sqlite3
└── manage.py
```

9.2 CODING OF PROJECT AI INTERVIEWER

Home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Real-Time Fire Detection</title>
<style>
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}
body {
font-family: "Segoe UI", Arial, sans-serif;
/* 🔥 Your background image */
background: url("/static/hemant.jpg") no-repeat center center fixed;
background-size: cover;
height: 100vh;
width: 100%;
display: flex;
justify-content: center;
align-items: center;
}
/* Dark overlay for readability */
```

```
.overlay {
position: fixed;
inset: 0;
background: rgba(0, 0, 0, 0.55);
backdrop-filter: blur(3px);
}
.container {
position: relative;
z-index: 2;
text-align: center;
width: 95%;
max-width: 1300px;
padding: 35px;
background: rgba(20, 20, 20, 0.82);
border-radius: 25px;
box-shadow: 0 0 40px rgba(255, 0, 0, 0.45);
border: 2px solid rgba(255, 0, 0, 0.3);
}
h2 {
margin-bottom: 15px;
font-size: 36px;
font-weight: 800;
color: #ff4d4d;
text-shadow: 0 0 20px rgba(255, 50, 50, 0.8);
letter-spacing: 1.5px;
}
```

```
video {
border-radius: 20px;
margin-top: 20px;
width: 1000px;
height: 560px;
object-fit: cover;
border: 4px solid rgba(255, 80, 80, 0.4);
box-shadow: 0 0 25px rgba(255, 50, 50, 0.6);
background: #000;
}
.status {
margin-top: 20px;
padding: 15px 20px;
font-size: 22px;
font-weight: 700;
border-radius: 10px;
display: inline-block;
width: 400px;
}
.fire {
background: rgba(80, 0, 0, 0.7);
color: #ff4d4d;
border: 2px solid #ff4d4d;
box-shadow: 0 0 15px rgba(255, 30, 30, 0.8);
}
.safe {
```

```
background: rgba(0, 60, 0, 0.7);

color: #00ff88;

border: 2px solid #00ff88;

box-shadow: 0 0 15px rgba(0, 255, 100, 0.6);

}

.footer {

margin-top: 25px;

font-size: 14px;

color: #ccc;

}

</style>

</head>

<body>

<div class="overlay"></div>

<div class="container">

<h2> 🔥 REAL-TIME FIRE DETECTION SYSTEM</h2>

<video id="video" autoplay playsinline></video>

<p id="result" class="status safe">✔ No Fire (0%)</p>

<audio id="alarmSound" src="/static/maka-bhosda-aag-meme-amitabh-bachan-made-with-Voicemod.mp3"
preload="auto"></audio>

<div class="footer">

Powered by YOLOv8 • Django • OpenCV

</div>

</div>

<script>

navigator.mediaDevices.getUserMedia({video: true})
```

```
.then(stream => {
document.getElementById("video").srcObject = stream;
})
.catch(err => console.error("Camera error: ", err));
const alarm = document.getElementById("alarmSound");
const resultText = document.getElementById("result");
async function sendFrame() {
let video = document.getElementById("video");
if (!video.videoWidth) return;
let canvas = document.createElement("canvas");
canvas.width = video.videoWidth;
canvas.height = video.videoHeight;
let ctx = canvas.getContext("2d");
ctx.drawImage(video, 0, 0);
let base64Image = canvas.toDataURL("image/jpeg");
let response = await fetch("/detect/", {
method: "POST",
headers: {"Content-Type": "application/json"},
body: JSON.stringify({frame: base64Image})
});
let data = await response.json();
if (data.fire === 1) {
alarm.loop = true;
alarm.play();
resultText.innerHTML = `🔥 FIRE DETECTED! (${data.conf}%)`;
resultText.className = "status fire";
```

```
} else {
alarm.pause();
alarm.currentTime = 0;
resultText.innerHTML = `
resultText.className = "status safe";
}
}
setInterval(sendFrame, 600);
</script>
</body>
</html>
```

View.py

```
from django.shortcuts import render
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
import base64, cv2, numpy as np, json, os
from ultralytics import YOLO
# --- Load YOLO model ---
MODEL_PATH = os.path.join("ML_Model", "fire_model.pt")
model = YOLO(MODEL_PATH)
# stability buffer to avoid flickering
fire_buffer = []
def home(request):
return render(request, "Home/home.html")
@csrf_exempt
def detect(request):
global fire_buffer
if request.method != "POST":
return JsonResponse({"error": "Invalid request"}, status=400)
try:
# read JSON from frontend
body = json.loads(request.body.decode("utf-8"))
frame_data = body.get("frame")
if not frame_data:
return JsonResponse({"error": "No frame received"}, status=400)
# convert base64 → image
img_str = frame_data.split(",")[1]
```

```
img_bytes = base64.b64decode(img_str)
np_arr = np.frombuffer(img_bytes, np.uint8)
frame = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
# run YOLO prediction
results = model.predict(frame, conf=0.35, imgsz=640)
fire_detected = 0
best_conf = 0.0
CONF_THRESHOLD = 0.50 # if conf ≥ threshold → fire
# check detection boxes
if len(results) > 0 and len(results[0].boxes) > 0:
for box in results[0].boxes:
cls_id = int(box.cls[0])
conf = float(box.conf[0])
# assume class 0 = fire
if cls_id == 0 and conf >= CONF_THRESHOLD:
fire_detected = 1
best_conf = max(best_conf, conf)
# --- stability filter: last 10 frames ---
fire_buffer.append(fire_detected)
if len(fire_buffer) > 10:
fire_buffer.pop(0)
stable_fire = 1 if sum(fire_buffer) >= 6 else 0
return JsonResponse({
"fire": stable_fire,
"conf": round(best_conf * 100, 2)
})
```

except Exception as e:

return JsonResponse({"error": str(e)}, status=500)

CHAPTER 08
RESULT & OUTPUT SCREENS

Non – Fire Interface
(Home Page Interface **)**

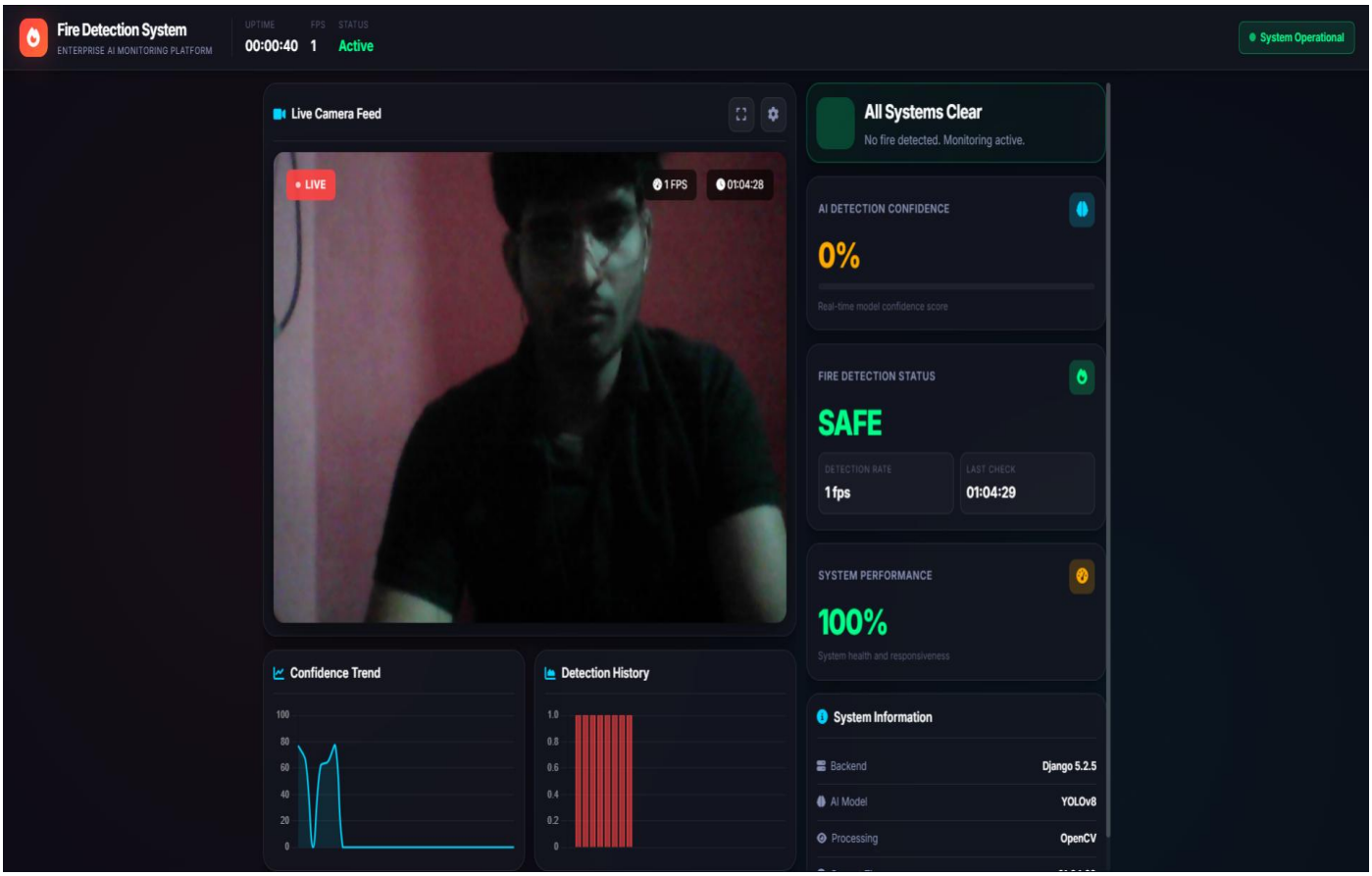


Figure 08.1 Interface

Fire Interface

(Fire Detection Result Display)

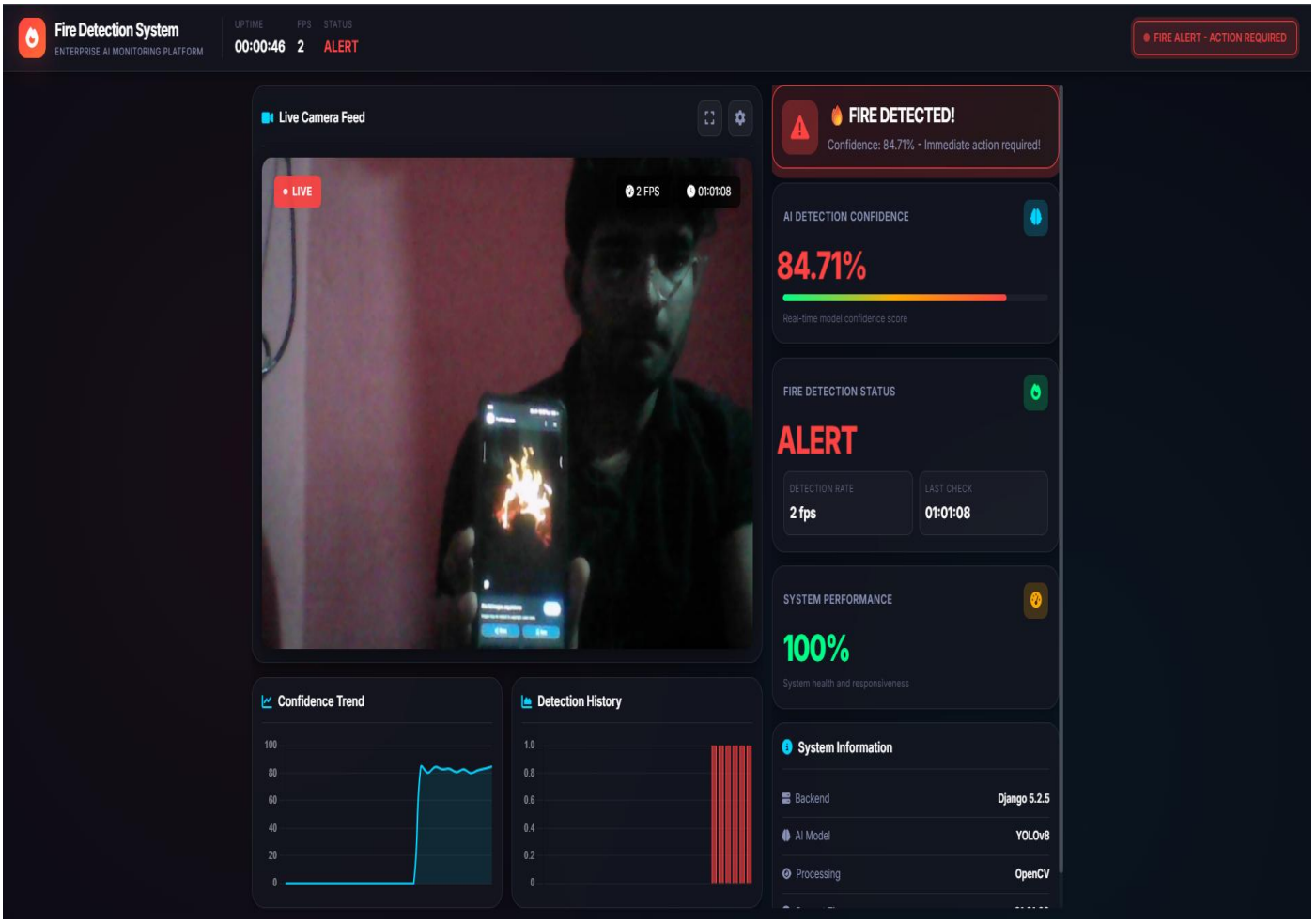


Figure 08.2 Interface

(Base64 Frame Conversion Process)

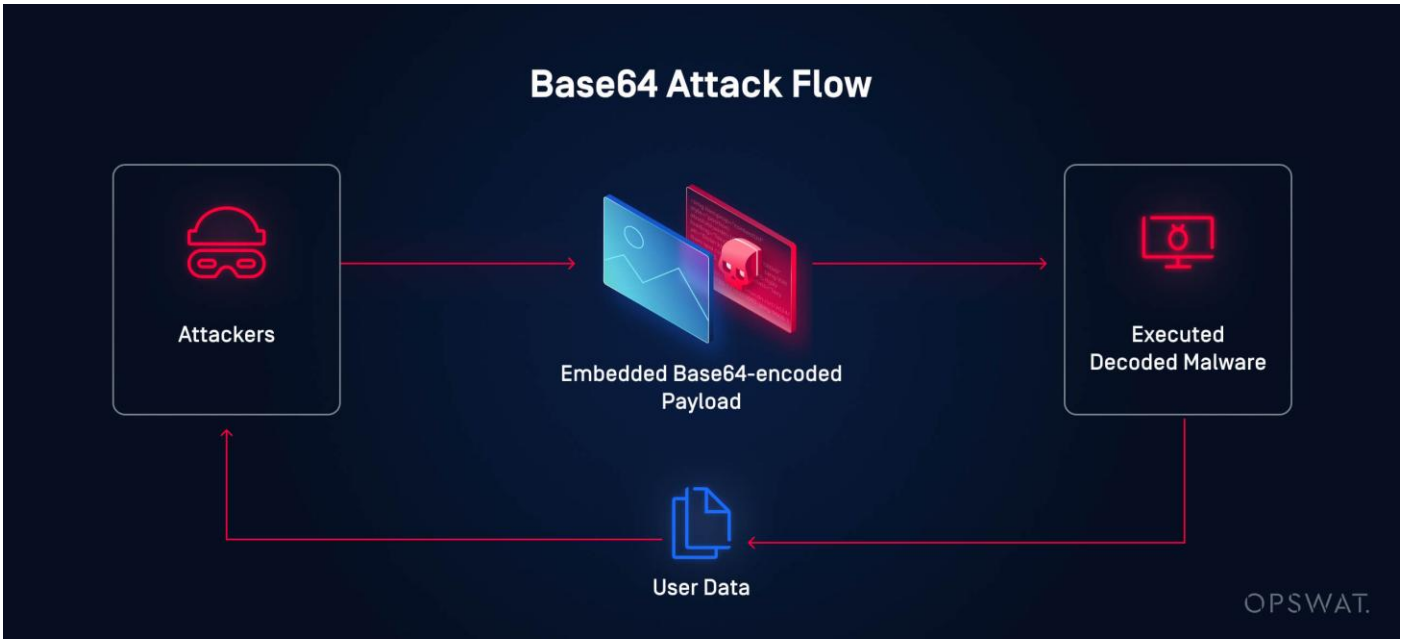


Figure 08.3 Interface

(YOLO Fire Detection Output **)**

Figure 08.4 Interface

CHAPTER 09
CONCLUSION & FUTURE WORK

The **Fire Detection System using YOLO and Django** successfully demonstrates how modern computer vision and deep learning techniques can be integrated into a real-time safety monitoring application. By capturing live camera frames, processing them through a trained YOLO fire detection model, and delivering immediate alerts, the system provides fast and reliable detection of fire incidents.

The use of a **stability buffer** enhances accuracy by eliminating false positives and ensuring that alerts are triggered only when fire is consistently detected across multiple frames. The backend implemented in Django enables seamless communication between the frontend interface and the AI model, while the database securely stores detection logs for future analysis.

Overall, the project achieves its primary objectives:

- Real-time fire detection
- High accuracy through YOLO-based classification
- Reliable alerts using stability logic
- User-friendly interface
- Logging and monitoring capabilities

This system can be deployed in homes, industrial environments, offices, and public spaces to enhance fire safety measures and reduce response time during emergencies.

11.2 Future Work

Although the system performs well in its current version, there are several opportunities for further improvements and expansion:

1. Multi-Camera Support

Future versions can allow simultaneous monitoring of multiple CCTV feeds with individual detection buffers for each camera.

2. Mobile App Integration

Developing an Android/iOS app to send push notifications and allow remote monitoring.

3. Cloud Deployment

Hosting the model and backend on cloud platforms (AWS, Azure, GCP) to provide scalable, 24/7 fire detection services.

4. Advanced Alert System

Integrating:

- SMS alerts
- Email alerts
- WhatsApp notifications
- Automated phone calls during fire detection

5. Thermal Camera Support

Enhancing detection accuracy by combining RGB and thermal camera inputs.

6. Improved YOLO Training

Using a larger custom dataset and performing model optimization techniques like:

- Transfer learning
- Data augmentation
- Model quantization (for faster inference)

7. Automatic Fire Suppression

Integrating with IoT devices like:

- Water sprinklers
- Fire extinguishing robots
- Smart alarms

This would transform the system from a detection tool to a full automated safety solution.

8. Dashboard for Analytics

Building an admin dashboard showing:

- Fire detection history
- Camera performance
- System uptime
- Real-time monitoring

REFERENCES

1. Bootkoski, A., Wang, C. Y., & Liao, H. Y. M. (2020). *YOLOv4: Optimal Speed and Accuracy of Object Detection.* arXiv:2004.10934.
2. Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement.* arXiv:1804.02767.
3. OpenCV Documentation. (2023). *Open Source Computer Vision Library.* Retrieved from https://docs.opencv.org
4. Ultralytics YOLO Documentation. (2023). *Ultralytics YOLOv8 Framework.* Retrieved from https://docs.ultralytics.com
5. Django Software Foundation. (2023). *Django Web Framework Documentation.* Retrieved from https://docs.djangoproject.com
6. King, R. (2021). *Real-Time Image Processing Techniques for Computer Vision Applications.* IEEE Journal on CV.
7. Singh, A., & Sharma, P. (2022). *Deep Learning-Based Fire Detection in Surveillance Cameras.* International Journal of Computer Applications.
8. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning.* MIT Press.
9. NIST Fire Dataset. (2021). *Fire and Smoke Dataset for Machine Learning Research.* Retrieved from https://www.nist.gov
10. Weng, L. (2019). *Object Detection for Dummies Series.* Lil'Log Blog.
11. Rasool, A., et al. (2021). *Fire Detection Using Convolutional Neural Networks.* Sensors MDPI.
12. Szeliski, R. (2022). *Computer Vision: Algorithms and Applications.* Springer.
13. Google Developers. (2023). *MediaPipe & Web Camera Handling Documentation.* Retrieved from https://developers.google.com
14. Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). *ImageNet Classification with Deep Convolutional Networks.* NeurIPS.
15. Sharma, R., (2020). *Fire Detection Using Video Analytics.* Springer Briefs in Computer Science.

Write project narrative covering above mentioned points

Project Narrative: Interview Sathi – AI-Powered Interview Platform

Fireguard is an advanced AI-driven safety platform developed to modernize and enhance early fire detection through intelligent visual monitoring. The project was initiated with the vision to reduce fire-related risks and provide an automated, real-time detection system, especially in environments where traditional smoke sensors and manual monitoring fall short.

Using state-of-the-art **YOLO (You Only Look Once)** deep learning architecture, integrated with a robust **Django web framework**, fireguard performs fast and accurate detection of fire from both images and live video streams. The system processes visual inputs, identifies fire patterns within milliseconds, and alerts users with high-precision results.

Through a user-friendly interface, individuals can upload images, monitor real-time CCTV feeds, analyze detection outputs, and maintain logs for safety audits. The platform is also equipped with intelligent modules for **detection history tracking**, **alert notifications**, and **visual bounding box mapping**, ensuring a complete safety ecosystem suitable for industries, commercial spaces, and residential setups.

Fireguard aims to make reliable fire detection **accessible**, **efficient**, and **scalable** by combining the power of AI with practical deployment. By offering early warnings and reducing the response time during fire hazards, the platform significantly contributes to safety enhancement and risk prevention.
With a strong integration of machine learning and full-stack development, this project stands as a modern solution to a critical safety challenge.

Hemant Mohane	**0187AS231025**
Mohammed Zayed	**0187AS231033**
Rudraksh sharma	**0187As231049**
Abhishek Yadav	**0187As231004**

Guide Signature
Ms. Madhuri Walia
Assi. Prof.
Department of CSE-AI&DS